

Deep Learning for NLP

Student name: Μπαρμπάρσος Θεοφάνης
sdi: sdi2200107

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1	Abstract	2
2	Data processing and analysis	2
2.1	Pre-processing	2
2.2	Data partitioning for train, test and validation	3
2.3	Tokenization	3
2.4	Vectorization	3
3	Algorithms and Experiments	3
3.1	Experiments	3
3.1.1	Tables of trials	5
3.2	Hyper-parameter tuning	7
3.3	Evaluation	7
3.3.1	Learning Curves	7
3.3.2	Confusion matrix	9
4	Results and Overall Analysis	9
4.1	Results Analysis	9
4.2	Comparison with the first project	9
4.3	Comparison with the second project	11
4.4	Comparison with the third project	11
5	Bibliography	12

1. Abstract

In this project, we developed a sentiment classifier by fine-tuning two state-of-the-art transformer-based models, BERT and DistilBERT, using PyTorch for the classification of English-language tweets. Leveraging the Twitter datasets from the previous assignments, this work explores the effectiveness of pre-trained language models in capturing nuanced sentiment patterns. The project involves meticulous fine-tuning of hyperparameters, regularization strategies, and training procedures to optimize performance while mitigating overfitting. We evaluated the models using comprehensive metrics, including accuracy, precision, recall, and F1-score, and validated their robustness through systematic experimentation. The implementation of both notebooks (using BERT and DistilBERT) were submitted via Kaggle, ensuring reproducibility with a fixed random seed. The results were analyzed and visualized to demonstrate the comparative strengths of BERT and DistilBERT in sentiment analysis tasks.

2. Data processing and analysis

2.1. Pre-processing

In this step, I implemented multiple preprocessing techniques to clean and normalize the Twitter dataset, ensuring it was suitable for feature extraction through tokenization-vectorization and classification. The following methods were applied:

1. **Lowercasing:** All text was converted to lowercase to ensure consistency and eliminate redundancy in token representation.
2. **Basic emojis handling:** Some very common emojis in a basic representation (e.g. :)) were replaced with representative key words such as happy, sad, love etc.
3. **Replacing Emails, URLs, and Mentions:** Using regular expressions, email addresses were replaced with the token "email", URLs with "url", and Twitter mentions (e.g., @username) with "username", as they do not contribute significantly to sentiment analysis.
4. **Handling Hashtags:** Hashtags were processed by removing the "#" symbol while preserving the associated word (e.g., "#happy" becomes "happy").
5. **HTML Entity Decoding:** HTML-encoded characters were converted to their corresponding textual representation (e.g., "&" becomes "&").
6. **Expanding Contractions:** Common English contractions were expanded to their full forms (e.g., "haven't" becomes "have not") to enhance textual clarity and standardization.
7. **Removing Non-Alphanumeric Characters:** Any character that is not an English letter, number, or whitespace was removed to eliminate noise and improve text quality.

2.2. Data partitioning for train, test and validation

In order to train, validate and test the model three datasets were given: `train_dataset`, `val_dataset`, `test_dataset`. In my implementation, the `train_dataset` was used to define the hyperparameters and the model's architecture, whereas the validation dataset was only used for the last training and evaluation. After this evaluation, the final model is used on the test dataset to output a submission file for the competition on Kaggle. The three datasets undergo the same pre-processing that includes all the techniques mentioned above along with tokenization and vectorization.

2.3. Tokenization

The textual data from all datasets underwent standardized tokenization using the BERT tokenizer (`bert-base-uncased`) and DistilBERT tokenizer (`distilbert-base-uncased`), which employ a WordPiece subword segmentation algorithm. This method splits text into subword units (e.g., "unhappiness" → ["un", "##happiness"]), effectively handling out-of-vocabulary (OOV) terms by decomposing them into known subword components. Special tokens, such as [CLS] (classification marker) and [SEP] (separator), were automatically inserted to indicate sentence boundaries, while the tokenizer also normalized inputs to lowercase and truncated/padded sequences to a fixed length (128 tokens). Attention masks were generated to distinguish actual tokens from padding, ensuring the model only processes relevant content. This approach balances vocabulary coverage and computational efficiency while preserving semantic granularity, even for rare or misspelled words. By leveraging subword units and contextual embeddings, the tokenization process captures deeper linguistic nuances, enabling the model to infer the exact meaning of words and generalize effectively to unseen vocabulary.

2.4. Vectorization

The tokenized sequences were mapped to dense vector representations via BERT's pre-trained embedding layers, which combine three types of embeddings: (1) token embeddings (768-dimensional vectors for each subword), (2) positional embeddings (encoding token order), and (3) segment embeddings (unused here, as single-sentence inputs were assumed). These embeddings were then processed through 12 transformer layers, where self-attention mechanisms dynamically refined contextual representations. For classification, the [CLS] token's final hidden state served as the aggregate sentence representation, capturing global context. Unlike static word embeddings (e.g., Word2Vec), BERT's contextualized vectors adapt to surrounding text, enabling nuanced disambiguation (e.g., differentiating "bank" in financial vs. river contexts). The model's end-to-end architecture eliminated the need for manual feature aggregation (e.g., mean-pooling), as the transformer inherently learned hierarchical patterns across tokens.

3. Algorithms and Experiments

3.1. Experiments

To tackle the sentiment analysis problem, I started by experimenting with different pre-processing techniques. Besides the ones mentioned above, some more complex techniques I tried were removing slang vocabulary (omg, lol, brb etc.), spell checking using libraries like `pyspellcheck`, stemming and lemmatization. In the end, I only kept the pre-processing I presented above, because as experiments showed, too complicated pre-processing wasn't beneficial for the results. Also we know that both BERT and DistilBERT are trained on informal or noisy data, typical of Twitter language, which means we don't need to go to extreme measures when it comes to text cleanup in order to aid the model's function. The hyperparameters that I experimented with in order to get the most out of the models are,

- the number of epochs
- the learning rate
- the weight decay

When it comes to the epochs, as discussed in the tutorials, 4 epochs were too many for the BERT model, something that was also confirmed by testing that led to overfitting. For DistilBERT, I implemented early stopping for 15 epochs maximum and with a patience of 2 epochs. In the end, the best number of epochs appears to be also 2, which makes sense based on the way DistilBERT works but I couldn't take it for granted just because it's the best for the BERT model. So I decided to experiment and test the BERT model for 2 and 3 epochs in the training process and DistilBert with early stopping before overfitting appears. As for the other two hyperparameters, I used Optuna to experiment and test a lot of possible combinations to try and converge to the optimal ones, that would provide the best performance of the models. It is important to note that, Optuna was minimizing the validation loss and this means that the best results on Kaggle were achieved using a bit different hyperparameters but that is of minimal significance as our goal is to maximize the performance of the models on any unknown set different than the training set(in this case, validation dataset).

3.1.1. Tables of trials.

- First, experiments for BERT with training for 3 epochs:(Table 1)

Trial	Learning Rate	Weight decay	Validation Loss
1	2.1002334012053967e-05	5.330543796461684e-06	0.43033312625198994
2	2.2350488419162358e-05	4.797887948273352e-05	0.43761448011926884
3	2.067449251297657e-05	7.972812913290453e-06	0.4308698641761856
4	6.831740123539786e-06	2.6338210907368616e-06	0.36997769100486105
5	1.3810849368128761e-06	0.00311185600997986	0.3821884366251388
6	2.3822125748859134e-06	8.32079253268283e-05	0.37437395272952206
7	4.8482724473194664e-06	0.003722915391743309	0.36793896957388467
8	2.0651810930706367e-06	5.48887309792033e-05	0.376283203324057
9	5.560149010338959e-06	0.004649485445995655	0.3678747267599376
10	1.0344673667593142e-06	2.177445950247307e-05	0.3881438763299078
11	1.1242122052652855e-06	3.9343021628520954e-05	0.3864902579334547
12	1.5819861827835263e-06	0.0009628567459775428	0.37960040284777585
13	3.141925726318445e-05	5.0814664513088425e-06	0.4729419161252818
14	2.0702946735266774e-06	1.382787990560754e-06	0.3748930691154498
15	7.616072425181388e-06	7.451775574994227e-06	0.37232355950575957
16	8.367305658389786e-06	0.002804027282190487	0.37473341466964416
17	1.6154941590054314e-05	4.1568033747614274e-05	0.4059895187307079
18	2.3590985131118113e-06	1.3430996913832895e-06	0.37330258098975666
19	9.702313297840714e-06	3.1888320940870305e-05	0.3788304212520707
20	1.1321870528351366e-06	2.387155092783356e-05	0.38639003103634095

Table 1: BERT experiments(3 epochs)

- Experiments for BERT with training for 2 epochs:(Table 2)

Trial	Learning Rate	Weight decay	Validation Loss
1	5.048637554530781e-06	1.0309720341082311e-05	0.3661916409348542
2	2.077170636710475e-06	0.00013125648631341912	0.381266564760568
3	1.1686355522341704e-06	0.008974578151358147	0.3941687400273557
4	1.7110437871926364e-06	0.009222386379142264	0.38423763533808153
5	2.353342001768476e-06	0.0016252003748842235	0.3791565848577697
6	2.6640341515362047e-05	4.1815767992551077e-05	0.370954057599576
7	8.440748351239059e-06	8.495141220208739e-06	0.3625098742685228
8	1.0830453708991054e-06	0.0002393820590952661	0.3961125877668273
9	2.7118637427050586e-06	0.00023342154167283564	0.3765754432835669
10	2.219247129698943e-06	1.2379704164402897e-06	0.37891020502684253
11	2.0221080610735814e-05	2.81578570409466e-05	0.36739556293442566
12	2.9855912215995474e-05	5.233234684239277e-06	0.37415080774786336
13	2.0809219613600006e-06	0.0011114758578285146	0.3817905103598001
14	8.881286312572462e-06	6.388873405617195e-05	0.3619096562659965
15	1.4178116578505638e-05	7.756065125360959e-05	0.3633313932115177
16	2.126835440190326e-05	1.770310804995584e-06	0.3676028702056633
17	1.7154934279118004e-06	4.9477103517199553e-05	0.38558092735848337
18	1.8832588218589612e-05	1.4098496852342177e-06	0.36690508380251113
19	4.157423366364046e-06	0.0008486648330565312	0.3685228884838662
20	4.2258565744499724e-05	0.00897628439218222	0.3781543184901183

Table 2: BERT experiments(2 epochs)

- Experiments for DistilBERT:(Table 3)

3.2. Hyper-parameter tuning

Ideally, more experiments could be done to capture an even wider variety of combinations but due to the assignment's time limitation and resources limitations from Kaggle, the above were made. However, we can extract a decent result out of them, as they are enough to see the model's best performance and behavior as we fine tune the hyperparameters. Ultimately, the best configuration was training for 2 epochs, with learning rate $8.881286312572462e-06$ and weight decay $6.388873405617195e-05$ for the BERT model and training for 2 epochs with a learning rate $1.293040592451508e-05$ and weight decay $2.1436761573115605e-05$. It must be made clear that these are the optimal combinations based on validation loss and these specific hyperparameters do NOT ensure the best performance on the test dataset because they were not chosen based on that performance. They do guarantee though, a very good result in both loss and accuracy on the test dataset and any unknown Twitter dataset for that matter.

3.3. Evaluation

The model's performance was assessed using standard evaluation metrics:

- **Accuracy:** Measures overall correctness of predictions.
- **Precision, Recall, and F1-score:** Provides insights into false positives and false negatives.
- **Confusion Matrix:** Visualizes model misclassifications.
- **Learning Curves:** Visualizes model's training and validation losses and accuracies throughout the epochs, thus providing helpful insight of the model's behavior.

3.3.1. Learning Curves.

- Learning curves for BERT for 2 epochs and the best hyperparameters:(Figure 1)
- Learning curves for BERT for 3 epochs and the best hyperparameters:(Figure 2)
- Learning curves for DistilBERT for 2 epochs and the best hyperparameters:(Figure 3)

Note that the validation loss line is actually not constant as the value decreases ever so slightly indicating absence of overfitting and achieving a decent loss from the first epoch. Specifically, validation loss of epoch 1 is: 0.3711 and validation loss of epoch 2 is: 0.3704.

Figure 1:

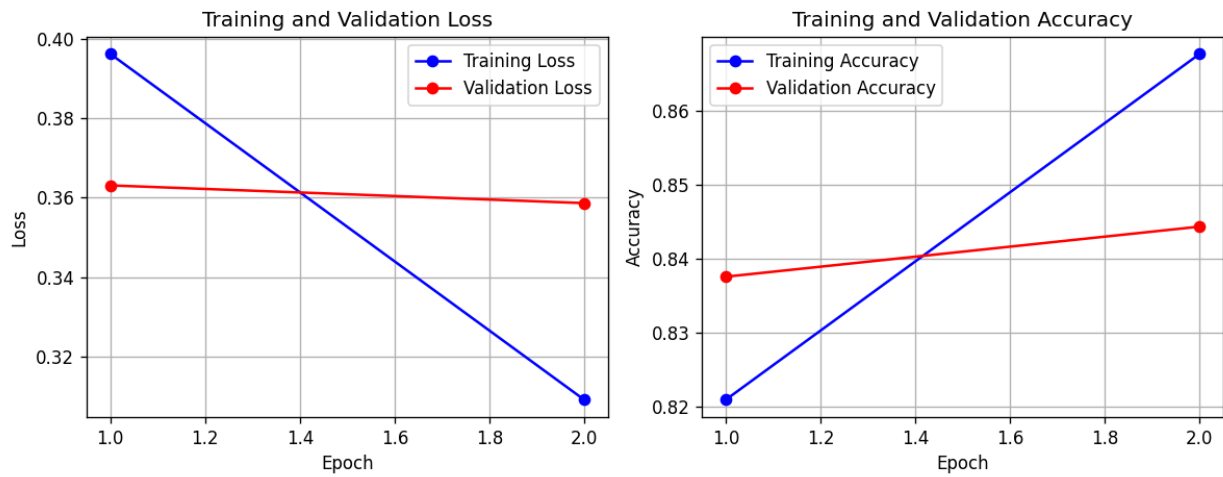


Figure 2:

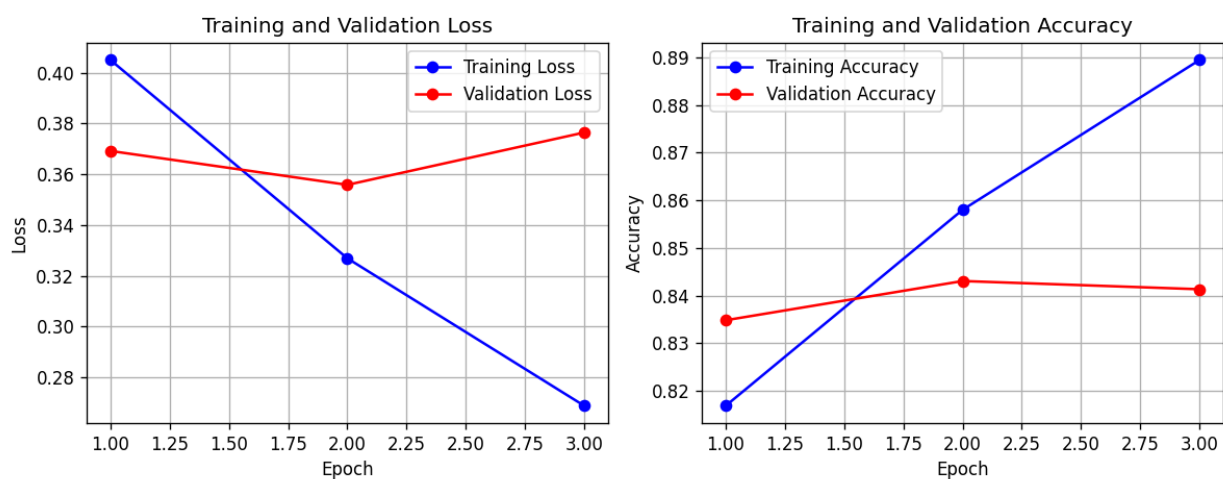
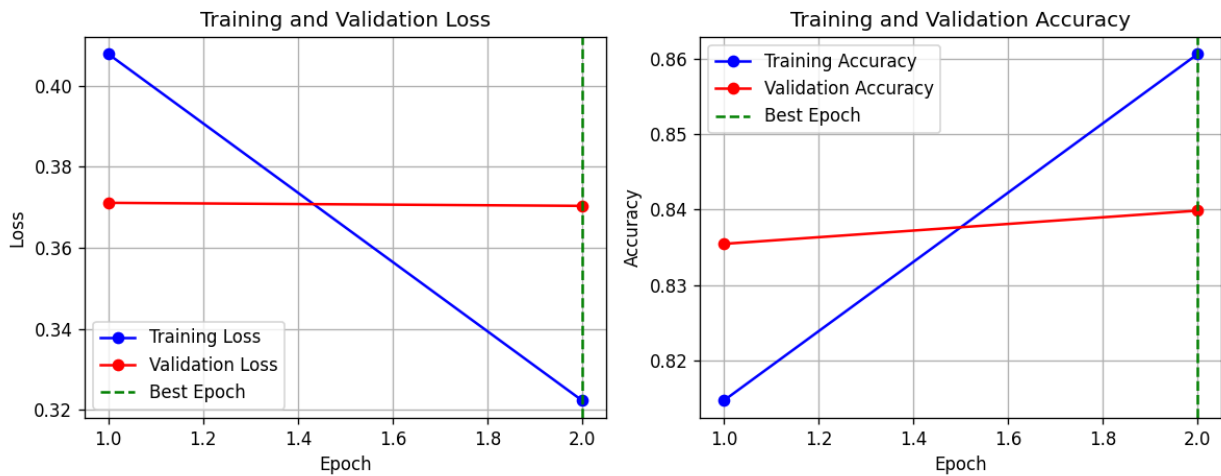


Figure 3:



3.3.2. Confusion matrix.

- Confusion matrix for BERT for 2 epochs and the best hyperparameters:(Figure 4)
- Confusion matrix for BERT for 3 epochs and the best hyperparameters:(Figure 5)
- Confusion matrix for DistilBERT for 2 epochs and the best hyperparameters:(Figure 6)

4. Results and Overall Analysis

4.1. Results Analysis

The results so far seem rewarding as both models seem to perform really well with almost all sets of hyperparameters, even better so, with the optimal combination. We came to the conclusion that for BERT the best number of epochs is 2, as 3 epochs of training would cause overfitting(as we see comparing Figures 1 and 2). Also for DistilBERT the best number of epochs is 2 as indicated by the best experiment. The two models's performances on the test datasets for the competitions on Kaggle are very good too with scores reached, 0.84584 and 0.83942, respectively.

4.2. Comparison with the first project

This project aimed at fine-tuning and experimenting with two pre-trained models to perform the task that we specifically wanted, in this case, sentiment analysis and classification whereas the first project focused on only one classification method through TF-IDF and Logistic Regression. These are much simpler approaches than BERT and DistilBERT but, apparently, the first project produced surprisingly good results, despite its naive approach. Of course, both models now are better than the first project and are better able to perform classification while extracting and utilizing deeper semantic meanings of words.

Figure 4:

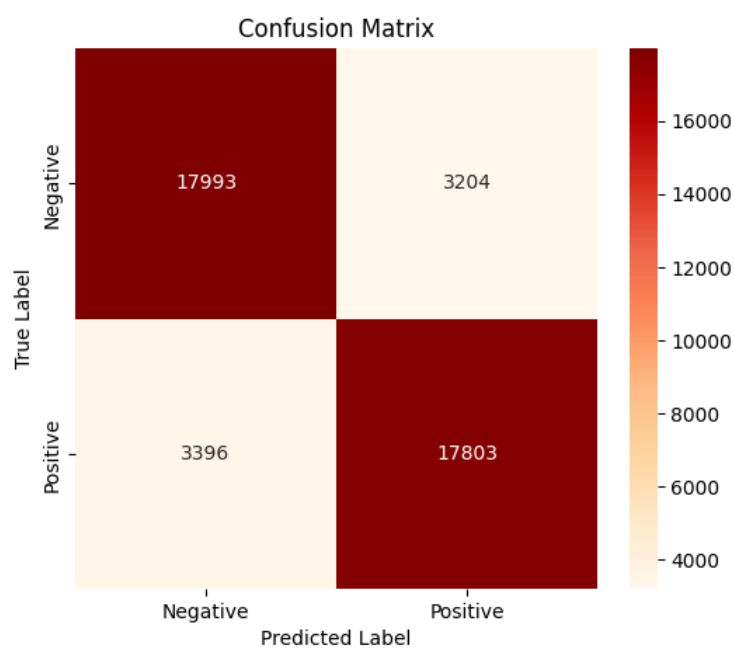


Figure 5:

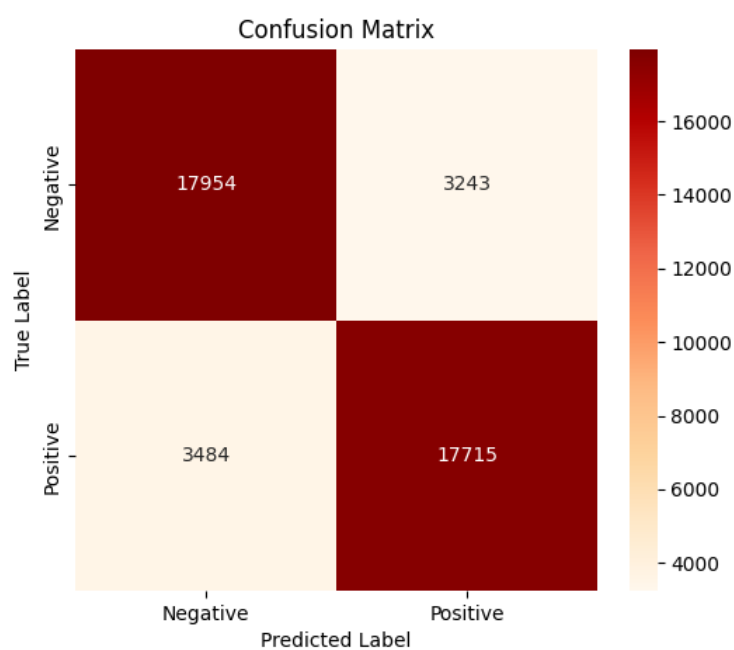
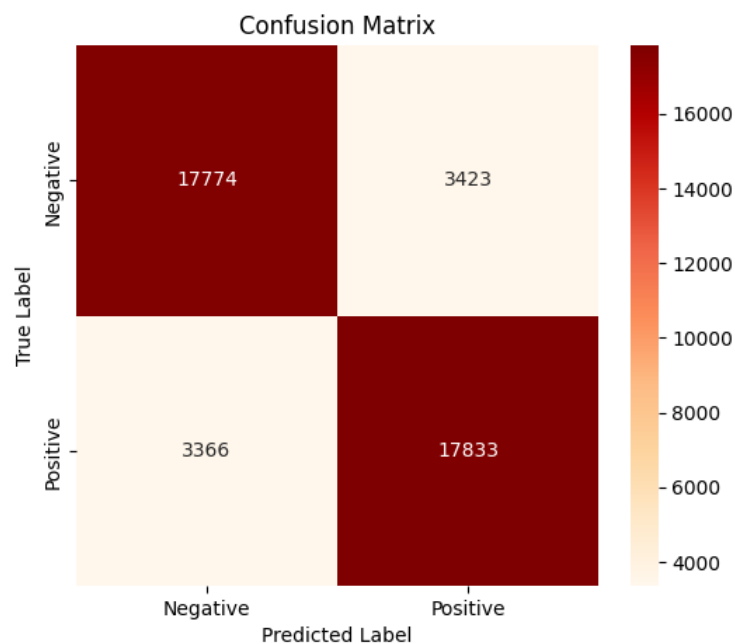


Figure 6:



4.3. Comparison with the second project

When compared to the second project, our models now, performed even better than the deep learning neural network we produced then and the two most important reasons for that are:

- Deep learning models often require more data and longer training to surpass pre-trained, complex models especially in the transformers category in text classification tasks. Logistic Regression, when paired with TF-IDF, is highly effective for short, sparse text like tweets.
- In the second project we used pre-trained Word2Vec embeddings, which, while rich, are not tailored to Twitter-specific language (slang, emojis, abbreviations), possibly weakening performance relative to TF-IDF's direct frequency-based representation.

The most important advantage of using BERT and DistilBERT is the way they extract and handle the meaning behind words. This allows them to distinguish and categorize natural language better, rather than only counting word frequency(project1) or trying to train a super complex, deep, neural network(project2).

4.4. Comparison with the third project

<Use only for project 4>

<Comment the results. Why the results are better/worse/the same?>

5. Bibliography

References

[1] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.

<You should link and cite whatever you used or "got inspired" from the web. Use the / cite command and add the paper/website/tutorial in refs.bib>
<Example of citing a source is like this:> [1] <More about bibtex>

Trial	Epochs	Learning Rate	Weight decay	Validation Loss
1	2	7.228535631317261e-06	0.0004111171289118429	0.37002159355384
2	2	1.5091389135862107e-05	0.0012653683204764843	0.3683344382891115
3	2	2.5798452731777197e-05	0.00010188997533514865	0.36942159732557694
4	2	7.301103759121131e-06	0.0006907976832579176	0.36979739306108006
5	2	5.060447975357406e-06	0.0002218099249473437	0.372800449183527
6	4	1.3267384888870186e-06	0.007831607922351038	0.38339250924452295
7	4	3.119807427535259e-06	0.0019245321115753187	0.3753475262477713
8	4	1.3272775964568894e-06	4.696145761050132e-06	0.3834010809659958
9	1	3.0227582838125792e-05	0.0004251349916594875	0.3746268356971021
10	2	1.7939633857360194e-05	0.00015560468153623372	0.3688889511857393
11	4	1.5416021213127252e-06	8.071834464413073e-05	0.38120884987543213
12	2	7.972688143184805e-06	6.1168126713945e-05	0.36944723102843985
13	4	2.3030747264574906e-06	8.026377009712585e-06	0.3766919429459662
14	4	3.8142695381222043e-06	0.0006199390553694661	0.3757913649307107
15	2	1.6609041928241528e-05	2.1726755721300897e-06	0.3686957742020769
16	2	4.0026810048876805e-05	1.4044672494073843e-06	0.37683957788179506
17	2	2.2147819926976767e-06	0.0002878546005816949	0.3770450159626187
18	2	5.722533835907149e-06	4.31247859569313e-06	0.37172148439119446
19	2	8.34893945430151e-06	1.6922800802881897e-05	0.3692927936562952
20	2	6.292519413748994e-06	0.0029583536375695618	0.37092818583519954
21	2	3.361050613982122e-05	0.003556579750022635	0.37425159209179426
22	2	1.0982272372522852e-05	1.6904013034362597e-05	0.36842194709013093
23	2	1.7011632583948192e-05	8.190214696386428e-06	0.36855250625677827
24	4	1.4297904584820136e-06	7.1053516386025605e-06	0.38224729020640535
25	2	2.9396226931741537e-05	0.0008045770271512412	0.37150164550205445
26	2	4.819484616967274e-05	3.463983432180653e-05	0.37876898255550634
27	4	2.518983119934732e-06	0.0017441479422189723	0.3760985025201204
28	2	1.876036621860828e-05	0.000789091860769746	0.36928764603048003
29	2	1.9634257144370012e-05	8.899191580002131e-05	0.3698860260506846
30	2	1.293040592451508e-05	2.1436761573115605e-05	0.36823900318932984
31	4	1.2758890956353847e-06	1.8427090595864374e-06	0.3840255624505709
32	2	5.864591707291439e-06	0.0007033470812998789	0.3715498957768926
33	2	5.843070182076244e-06	5.235767503059212e-05	0.37140574334927323
34	4	3.3742835341892782e-06	0.0003274422925871465	0.37538009373646863
35	2	5.402285540012536e-06	1.1873061710344042e-05	0.3722111657486772
36	2	8.707617725889813e-06	1.0469056769534236e-06	0.3691616789574893
37	2	7.796289766184418e-06	0.00021758532220212494	0.3696067002359426
38	2	1.0177793044169787e-05	0.006705585000435287	0.36853946785319525
39	2	3.941303647704983e-05	4.342581168876263e-05	0.37651513200885844
40	2	1.3585372739011365e-05	1.813090007741068e-06	0.3681652431330591

Table 3: DistilBERT experiments